Практикум 1. Инструменты

Ollama, Protégé и мини-агент: как поставить, как пользоваться, как построить

От интеллектуального поиска к интеллектуальным агентам

О. М. Атаева А. П. Халов

ВМК МГУ, осень 2026

AI4Science AI4Research AI4Math

Три инструмента и зачем каждый

Ollama

Языковая модель на своём компьютере. Нужна, чтобы агент работал без ключа и без отправки данных наружу. Используется в практикуме и в заданиях с агентом.

Protégé

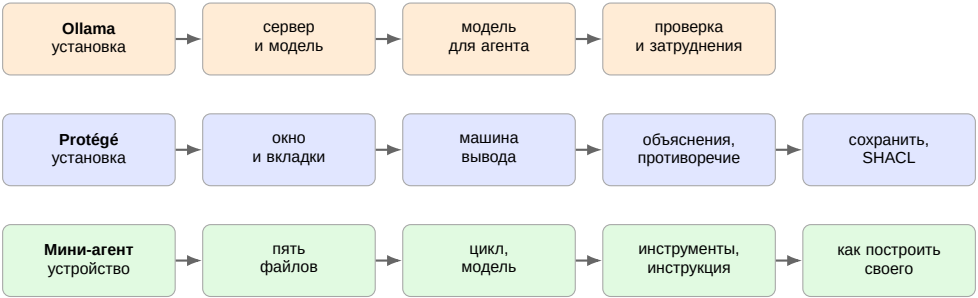
Редактор онтологий. Нужен, чтобы видеть классы, свойства и индивидов, запускать машину вывода и читать её объяснения. Задания 2 и 3.

Мини-агент

Программа из пяти файлов, которую курс собирает с первой лекции. Нужна, чтобы понять, из чего состоит агент, и построить своего над своей областью.

- Три части презентации, по одной на инструмент. Каждая отвечает на три вопроса: что это, как поставить, как пользоваться.
- Окна программ показаны схемами по документации и по проверенным командам. Настоящие снимки экрана добавляются в папку `img/screens/real/`.
- Все команды и файлы лежат в папке `practice1/`.

Карта



- Части независимы. Ollama нужна для живого агента, Protégé для заданий по онтологии, мини-агент для понимания и для своего проекта.
- Порядок установки на своей машине: Python-пакеты, затем Protégé, затем Ollama.

Часть 1. Ollama

Языковая модель на своём компьютере

Что такое Ollama и зачем она в курсе

Что это

- Программа, которая скачивает модели с открытыми весами и запускает их на вашем компьютере.
- Поднимает сервер на адресе `http://127.0.0.1:11434`.
- Отвечает на запросы в формате OpenAI: тот же код, что для облачной модели.

Зачем в курсе

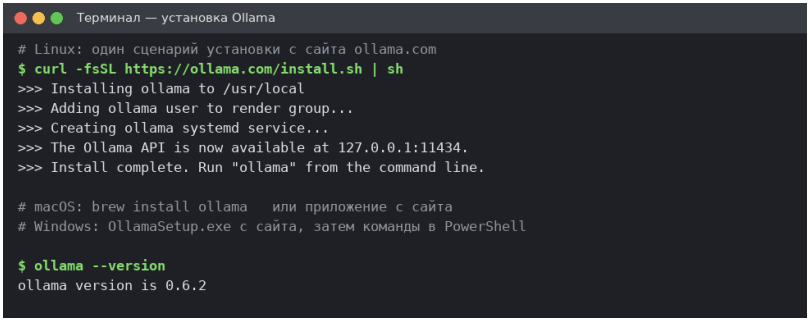
- Агент работает без ключа и без оплаты.
- Корпус и данные остаются на вашей машине.
- Один и тот же агент переключается между локальной и облачной моделью тремя переменными окружения.



Главное

Ollama даёт модели тот же адрес и формат, что у облачных сервисов. Код агента не знает, где модель работает.

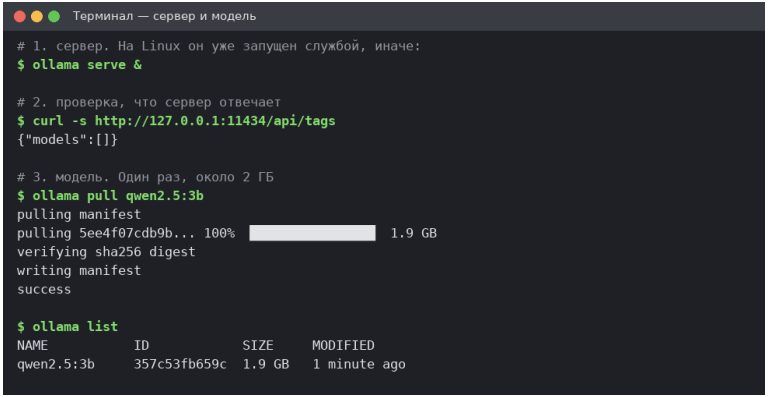
Установка



- Linux: один сценарий с сайта `ollama.com`. Он ставит программу и службу, сервер запускается сам.
- macOS: приложение с сайта или `brew install ollama`. Windows: установщик `OllamaSetup.exe`.
- Проверка: `ollama --version`. На машине курса стоит версия 0.6.2.

схема окна по документации, не снимок экрана

Сервер и модель



- Сервер отвечает на `/api/tags` списком моделей. Пустой список означает: сервер есть, моделей нет.
- Модель скачивается один раз командой `ollama pull`. Для курса достаточно `qwen2.5:3b`: около 2 ГБ, умеет вызывать инструменты.

схема окна по документации, не снимок экрана

Первый разговор

Терминал — первый разговор

```
$ ollama run qwen2.5:3b
>>> Ответь одним словом: какая столица Франции?
Париж.

>>> /bye

# то же через OpenAI-совместимый адрес, из Python
$ python3 -c "
from openai import OpenAI
c = OpenAI(api_key='ollama', base_url='http://127.0.0.1:11434/v1')
r = c.chat.completions.create(model='qwen2.5:3b',
      messages=[{'role': 'user', 'content': 'Ответь одним словом: готов?'}])
print(r.choices[0].message.content)"
Готов.
```

- `ollama run` открывает диалог в терминале. Выход: `/bye`.
- Из Python модель вызывается библиотекой `openai` с адресом сервера. Ключ любой непустой.

схема окна по документации, не снимок экрана

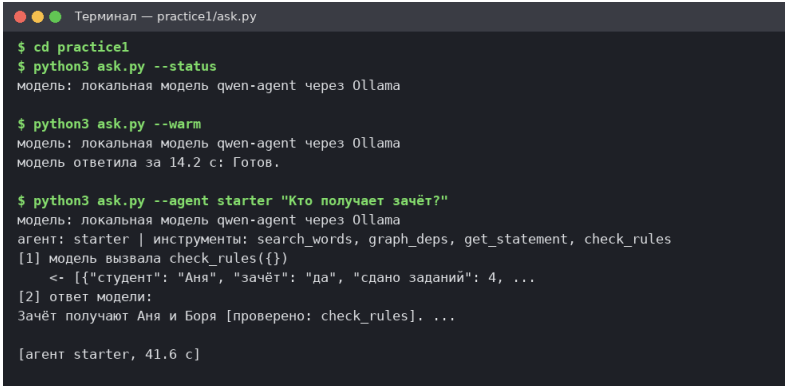
Модель для агента: окно контекста



- У Ollama 0.6.2 окно контекста по умолчанию 2048 токенов. Описания инструментов и результат одного вызова в него не помещаются.
- `Modelfile` собирает производную модель `qwen-agent` с окном 8192 и температурой 0. Команда `create` ничего не скачивает.

схема окна по документации, не снимок экрана

Проверка из папки практикума



- `ask.py --status` говорит, какую модель нашёл. `--warm` загружает её в память первым коротким запросом.
- Время ответа печатается в конце. Оно и определяет, что показывать вживую.

схема окна по документации, не снимок экрана

Что пошло не так

Терминал — что пошло не так

```
$ ollama serve
Error: listen tcp 127.0.0.1:11434: bind: address already in use
# сервер уже запущен службой. Ничего делать не нужно

$ python3 ask.py --status
модель: Ollama отвечает, модели qwen-agent нет (есть: qwen2.5:3b). Показываем записи
# не собрана производная модель: ollama create qwen-agent -f Modelfile

$ python3 ask.py "вопрос"
[1] модель вызвала graph_deps({'wrong': 1})
    <- {"ошибка": "TypeError: graph_deps() got an unexpected keyword argument 'wrong'"}
# маленькая модель ошиблась в аргументе. Ошибка вернулась ей, цикл продолжается

# ответ дольше минуты: модель на процессоре. Прогреть заранее: python3 ask.py --warm
# держать в памяти: OLLAMA_KEEP_ALIVE=2h ollama serve
```

схема окна по документации, не снимок экрана

Сколько ждать и что выбрать

Модель	На диске	Память	Чего ждать
qwen2.5:3b	1,9 ГБ	4 ГБ ОЗУ	на процессоре 10--60 с на шаг агента. Иногда ошибается в формате вызова
qwen2.5:7b	4,7 ГБ	8 ГБ ОЗУ или видеокарта 8 ГБ	заметно надёжнее вызывает инструменты. На процессоре медленно
qwen2.5:14b	9 ГБ	видеокарта 16 ГБ	близко к облачным моделям по надёжности

- Модель должна поддерживать вызов инструментов: это указано в её карточке на сайте Ollama словом «tools».
- Другая модель подключается заменой строки `FROM` в `Modelfile` и повторным `ollama create`.
- Сервер факультета или облачный ключ подключаются тремя переменными: `LLM_API_KEY`, `LLM_BASE_URL`, `LLM_MODEL`. Код не меняется.

Где это в материалах курса

Файл	Что делает
`practice1/Modelfile`	производная модель `qwen-agent` с окном 8192
`practice1/ask.py`	вопрос модели из терминала: без инструментов и с тремя агентами
`practice1/llm_local.py`	выбор модели: переменные окружения, затем локальная Ollama, иначе записи
`lecture2/agent/run_local.sh`	проверка готовности и прогон двух агентов с сохранением протоколов
`practice1/ПОДГОТОВКА_к_показу.md`	порядок действий за день и за 15 минут до занятия
`practice1/tests/fake_server.py`	подставной сервер для репетиции без модели

Главное

Без Ollama всё в курсе работает на записях. С Ollama те же файлы работают вживую.

Часть 2. Protégé

Редактор онтологий и машина вывода

Что такое Protégé и зачем он в курсе

Что это

- Редактор онтологий Стэнфордского университета, свободный, на Java.
- Открывает файлы OWL в любом синтаксисе: RDF/XML, Turtle, манчестерский.
- Встроенные машины вывода, расширения для правил SWRL.

Зачем в курсе

- Увидеть онтологию, записанную в блокноте: иерархию, свойства, индивидов.
- Запустить машину вывода и прочитать выведенные факты и объяснения.
- Получить противоречие и найти его участников.
- Проверить и сдать `kg/ontology.ttl`.

- Писать онтологию можно и без Protégé, из Python. Protégé нужен, чтобы смотреть и проверять.
- Проверка по формам SHACL в Protégé без расширения не делается. Для неё используется pySHACL из терминала.

Установка и запуск

Терминал — запуск Protégé

```
# скачать Protégé Desktop 5.6 с protege.stanford.edu, распаковать
$ unzip Protege-5.6.4-linux.zip && cd Protege-5.6.4
$ ./run.sh
# macOS: Protégé.app,  Windows: Protege.exe. Java входит в поставку

# открыть онтологию лекции: File → Open → lecture2/data/review.owl
# сохранить в Turtle: File → Save as → Turtle Syntax → kg/ontology.ttl
```

- Сайт `protege.stanford.edu`, раздел Protégé Desktop, версия 5.6 или новее. Архив для своей системы.
- Java входит в поставку, отдельно ставить её не нужно.
- Первый запуск занимает около минуты. Окно открывается на вкладке Active ontology.

схема окна по документации, не снимок экрана

Окно и вкладки

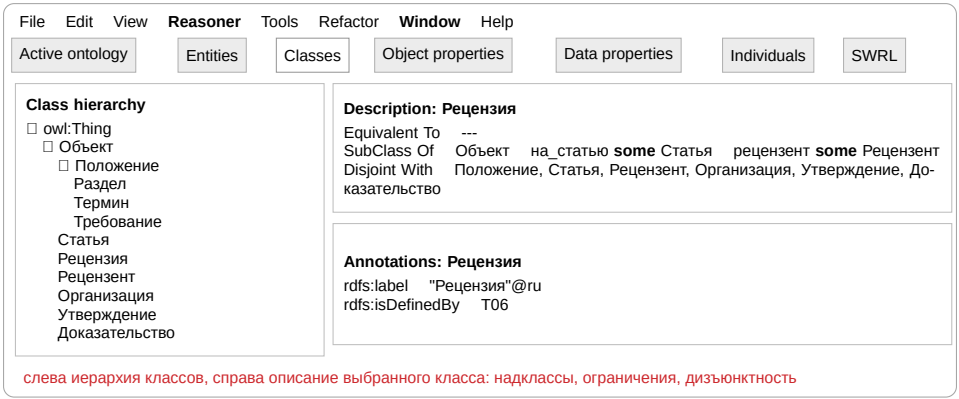


схема окна по документации, не снимок экрана

Открыть онтологию лекции

Вкладка	Что видно	На примере `review.owl`
Active ontology	число классов, свойств, индивидов, аксиом	12 классов, 10 объектных свойств, 69 индивидов
Classes	иерархия. Для выбранного класса: надклассы, ограничения, непересекающиеся классы	у класса Рецензия два ограничения `some`
Object properties	области и характеристики	у `зависит_от` отмечено Transitive
Data properties	свойства данных и типы	`оценка`: decimal, `сдано`: date
Individuals	индивиды, их типы и факты	R65: тип Требование, четыре факта `зависит_от`
SWRL	правила	правило для требования 6.2

- `File → Open`, файл `lecture2/data/review.owl` или свой `kg/ontology.ttl`.
- Русские имена показываются по идентификатору. Чтобы видеть подписи: `View → Render by label`.
- Если вкладки SWRL нет: `Window → Tabs → SWRLTab`. Если её нет и там: `File → Check for plugins`, установить SWRLTab.

Вкладка Object properties

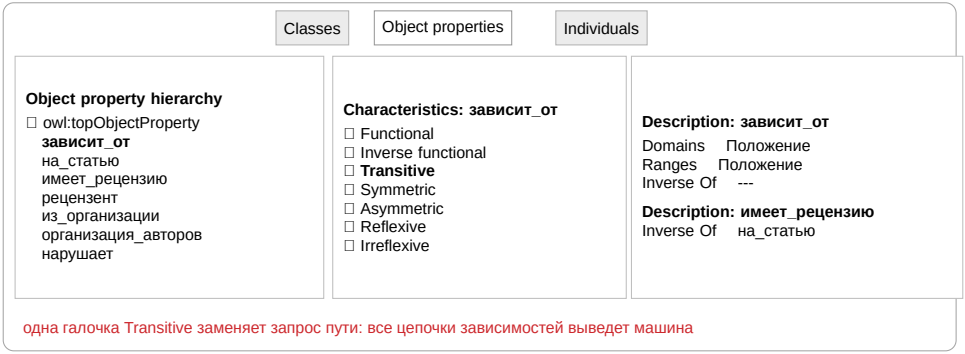


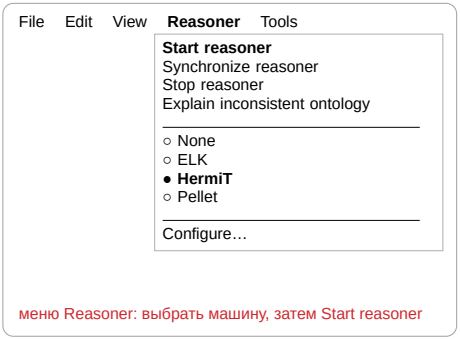
схема окна по документации, не снимок экрана

Вкладка Individuals



схема окна по документации, не снимок экрана

Машина вывода: запуск



меню Reasoner: выбрать машину, затем Start reasoner

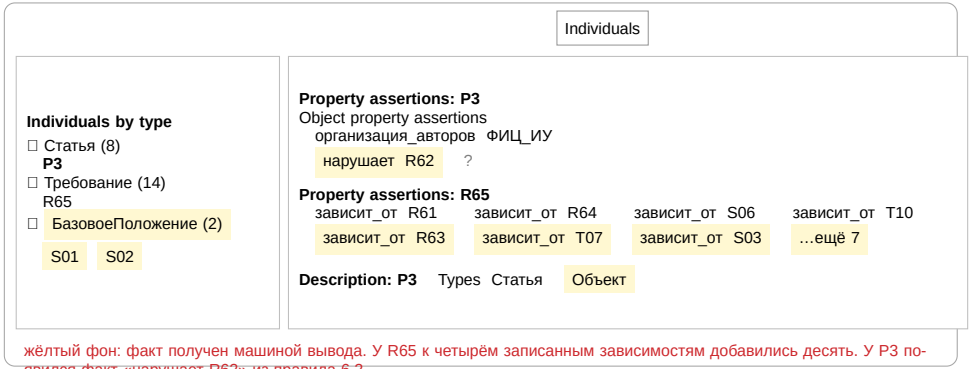
схема окна по документации, не снимок экрана

Порядок действий

1. Меню `Reasoner`: выбрать машину. HermiT входит в поставку. Pellet ставится через `File → Check for plugins`, он нужен для объяснений противоречия в том виде, как в лекции.
2. `Reasoner → Start reasoner`. Строка состояния внизу показывает, что вывод выполнен.
3. После каждой правки онтологии: `Reasoner → Synchronize reasoner`. Без этого выводы устаревают.

Правила SWRL исполняют Pellet и HermiT. Встроенный ELK правила пропускает.

Выведенное показывается жёлтым



жёлтый фон: факт получен машиной вывода. У R65 к четырём записанным зависимостям добавились десять. У P3 появился факт «нарушает R62» из правила 6.2

схема окна по документации, не снимок экрана

Объяснение вывода

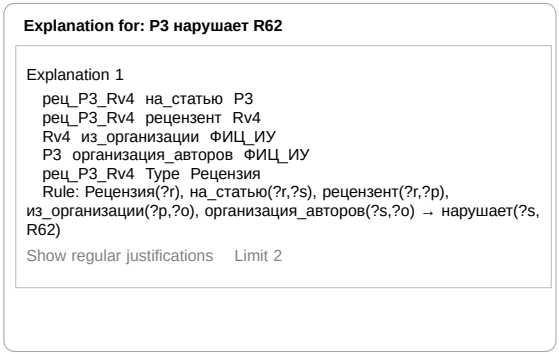


схема окна по документации, не снимок экрана

- Знак вопроса рядом с жёлтым фактом открывает окно объяснения.
- В нём перечислены аксиомы и факты, из которых факт следует: четыре строки журнала и правило.
- Для зависимости R65 от S03 объяснение покажет цепочку фактов `зависит_от` и аксиому транзитивности.

Это те же уровни объяснения, что в лекции 1: правило и свидетельство.

Противоречие

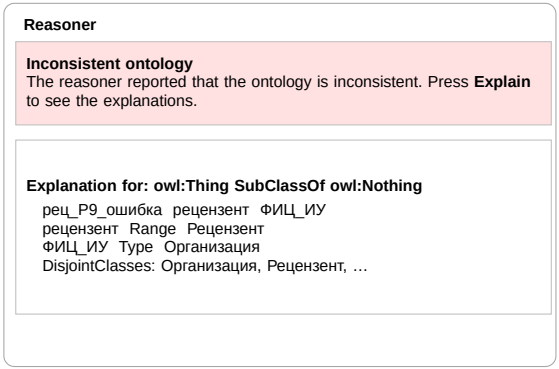


схема окна по документации, не снимок экрана

Как получить

1. На вкладке Individuals создать индивид `рец_P9_ошибка` типа Рецензия.
2. Добавить факт `рецензент` → `ФИЦ_ИУ`.
3. `Reasoner` → `Synchronize reasoner`. Появится окно о противоречии.
4. `Explain`: четыре участника, как в лекции 2.
5. Удалить факт, синхронизировать снова.

Для задания противоречие вносится в копию файла. Сдаваемый файл остаётся чистым.

Неудовлетворимый класс и выводимые классы

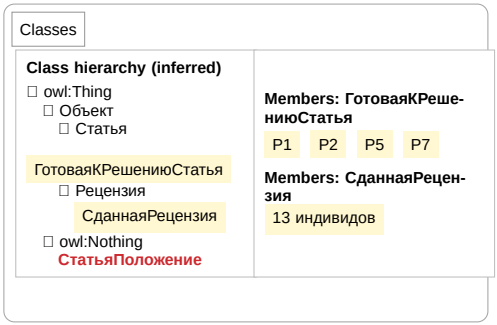


схема окна по документации, не снимок экрана

- Кнопка `Class hierarchy (inferred)` показывает иерархию после вывода. Выводимые классы встают на место по своим определениям.
- Класс красным под `owl:Nothing`: неудовлетворимый. У него не может быть экземпляров. Это ошибка проектирования, лекция 3.
- Члены выводимого класса жёлтые: их отнесла туда машина. Класс с условиями «все» и «нет» наполняется только после замыкания мира, в Protégé это делается добавлением аксиом вручную или из блокнота.

Сохранить и проверить формами

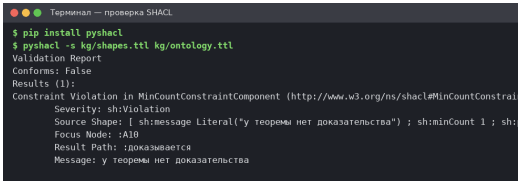
Сохранить

- `File → Save as`. Формат Turtle Syntax для сдачи: `kg/ontology.ttl`.
- Формат RDF/XML для обмена с Owlready2.
- Выведенные факты в файл не попадают: сохраняется записанное. Чтобы сохранить выводы, `File → Export inferred axioms as ontology`.

Частые затруднения

- Выводы не обновились: нужен Synchronize reasoner.
- Правило не сработало: выбран ELK. Переключить на HermiT или Pellet.
- Имена по идентификатору: View → Render by label.

схема окна по документации, не снимок экрана



Проверка по формам делается из терминала: `pyshacl -s kg/shapes.ttl kg/ontology.ttl`. Отчёт называет узел, свойство и сообщение формы.

Protégé в заданиях

Задание	Что сделать в Protégé	Что записать
2 (лекция 2)	открыть `kg/ontology.ttl`, проверить иерархию и свойства, запустить машину вывода, внести ошибку в копию и прочитать объяснение	выведенный факт и разбор противоречия в `kg/ONTOLOGY.md`
3 (лекция 3)	после пересмотра по вопросам: inferred hierarchy, члены выводимых классов, нет ли красных классов	лог машины вывода в `kg/VALIDATION.md`
любое	сохранить как Turtle	`kg/ontology.ttl` в репозитории

Главное

Protégé показывает то, что блокнот делает кодом. Один файл, два окна на него.

Часть 3. Мини-агент

Из чего он состоит и как построить своего

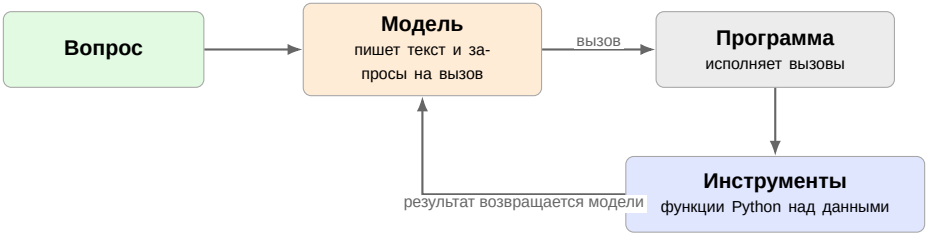
Откуда взялся мини-агент

- **Лекция 1, шаг 5 демонстрации.** Четыре функции, которые запускались вручную, отданы модели: поиск по словам, путь по графу, правила над журналом, внешняя проверка. Модель сама выбирает, что вызвать.
- **Лекция 2, шаг 10.** Те же цикл и модель, другие инструменты: машина вывода и проверка непротиворечивости. Правило 6.2 ушло из кода в онтологию.
- **Лекция 3, шаг 10.** Добавлен инструмент проверки данных по формам и пометка «не прошло проверку».
- **Разбор.** Файл `13_РАЗБОР_агента_Л1_Л2`: агент вынесен из блокнотов в пять самостоятельных файлов. Они лежат в `lecture2/agent/` и в `practice1/`.

Главное

На всех лекциях агент один и тот же. Меняются только инструменты и инструкция.

Что такое агент



1. Модель получает вопрос и описания инструментов.
2. Модель пишет текст. Часть текста может быть запросом: имя инструмента и аргументы.
3. Программа выполняет запрос и возвращает результат модели текстом.
4. Шаги 2 и 3 повторяются, пока модель не ответит без запросов. Это ответ.

Главное

Модель ничего не запускает сама. Она просит. Запускает программа, и только те функции, которые в неё положили.

Шесть компонентов и где они в коде

	Компонент	Что это	Где в коде
p_θ	модель	порождает текст и вызовы	`openai_llm`, `scripted_llm` в `agent_core.py`
D	ведущее приложение	ведёт цикл, исполняет вызовы	`run_agent` в `agent_core.py`
T	инструменты	допустимые действия	функции, список `TOOLS`, словарь `IMPL`
P	контекст и инструкции	роль, правила, сведения	строка `SYSTEM`, описания в `TOOLS`, результаты инструментов
G	цель	что считается результатом	`QUESTION` и требования к виду ответа
V	верификатор	внешняя проверка	`check_rules`, машина вывода, проверка по формам

- Модель сменная: за ней одна функция с двумя аргументами.
- Верификатор стоит снаружи модели: модель вердикты только пересказывает и помечает.

Пять файлов

```
practice1/
├── data/          три файла CSV, review.owl
├── agent_core.py   ядро: модель и цикл
├── search_bm25.py  поиск по словам
├── agent_l1.py     агент лекции 1
├── agent_l2.py     агент лекции 2
└── starter/src/
    └── agent_my.py агент над своим корпусом
```

Файл	Зависит от области?
`agent_core.py`	нет. Переносится как есть
`search_bm25.py`	нет. Любой корпус с текстами
`agent_l1.py`	да. Задача редактора: журнал, положение
`agent_l2.py`	да. Онтология редактора
`agent_my.py`	да. Образец для своего корпуса

- Ядро не меняется при переходе от лекции 1 к лекции 2 и к лекции 3. Меняются файлы с инструментами.
- Запуск без модели: `python3 agent_l1.py`. Инструменты вызываются по-настоящему, ответ модели берётся из записи.

Разговор как список сообщений

Роль	Кто пишет	Что содержит
`system`	автор агента	инструкция
`user`	пользователь	вопрос
`assistant`	модель	текст или запросы на вызов
`tool`	программа	результат одного вызова

Настоящий след, два шага

0. `system`: «Ты ассистент редактора…»
1. `user`: «Нарушает ли статья РЗ…?»
2. `assistant`: вызов `check_consistency{}`
3. `tool`: `{"непротиворечива": true}`
4. `assistant`: вызов `infer{"article": "РЗ"}`
5. `tool`: `{"выведено": […]}`
6. `assistant`: текст ответа

- При каждом обращении модель получает весь список целиком. Памяти между обращениями у неё нет.
- Результат инструмента есть текст: словарь Python превращается в строку JSON.
- Порядок вызовов выбирает модель. У живой модели он может отличаться от записи.

Ядро: цикл `run_agent`

```
def run_agent(question, llm, system, tools, impl, max_steps=8):
    messages = [{"role": "system", "content": system},
                {"role": "user", "content": question}]
    for step in range(1, max_steps + 1):
        m = llm(messages, tools)          # модель решает
        messages.append(m)
        calls = m.get("tool_calls") or []
        if not calls:                     # вызовов нет: ответ
            print(m["content"]); return messages
        for tc in calls:                  # программа исполняет
            name = tc["function"]["name"]
            args = json.loads(tc["function"]["arguments"] or "{}")
            try:
                result = impl[name](**args)
            except Exception as e:        # ошибка идёт модели
                result = {"ошибка": f"{type(e).__name__}: {e}"}
            messages.append({"role": "tool", "tool_call_id": tc["id"],
                    "content": json.dumps(result, ensure_ascii=False)})
```

Три детали

- `max_steps`: модель может зациклиться. Ограничение гарантирует завершение.
- `try`: несуществующий инструмент или неверный аргумент не роняют программу. Ошибка возвращается модели, и на следующем шаге она может исправиться.
- Печать обрезана до 160 знаков. В `messages` результат целиком.

Двадцать строк. Это весь агент как цикл.

Модель: две функции одного вида

Настоящая модель

```
def openai_llm(messages, tools):
    client = OpenAI(api_key=os.environ["LLM_API_KEY"],
                    base_url=os.environ.get("LLM_BASE_URL"))
    r = client.chat.completions.create(
        model=os.environ.get("LLM_MODEL"),
        messages=messages, tools=tools)
    return r.choices[0].message.model_dump(
        exclude_none=True)
```

Любой сервер с OpenAI-совместимым API: Ollama, сервер факультета, облако. Три переменные окружения.

Модель-заглушка

```
def scripted_llm(plan, answer):
    steps = iter(plan)
    def llm(messages, tools):
        step = next(steps, None)
        if step is None:
            return {"role": "assistant",
                    "content": answer}
        name, args = step
        return {"role": "assistant", "content": None,
            "tool_calls": [{"id": ..., "type": "function",
                  "function": {"name": name,
                    "arguments": json.dumps(args)}}]}
    return llm
```

Выдаёт вызовы из плана по одному, затем записанный ответ. Нужна, чтобы проверять инструменты и цикл без модели.

Главное

У обеих функций один вид: сообщения и описания на входе, одно сообщение на выходе. Цикл не знает, какая из них ему передана.

Инструмент есть функция

```
def graph_deps(doc_id: str):
    """На какие положения транзитивно
    опирается документ."""
    q = ("PREFIX ex: <http://example.org/kg#> "
         f"SELECT ?dep WHERE {{ ex:{doc_id} "
         "ex:dependsOn+ ?dep . }}")
    ids = sorted(str(r.dep).split("#")[1]
                 for r in g.query(q))
    return {"документ": doc_id,
            "опирается_на": [f"{i} {title[i]}"
                             for i in ids]}
```

Правила для инструмента

- Обычная функция с понятными именами аргументов.
- Возвращает словарь или список из простых значений.
- При неверном аргументе возвращает `{"ошибка": "..."}`.
- Результат короткий и самоописанный: идентификаторы и названия.
- Сведение идёт вместе с основанием: источник, правило, путь.
- Только читает данные. Ничего не меняет в файлах.

Главное

Инструменты агента первых лекций есть те же функции, что запускались вручную в демонстрации.

Описание инструмента для модели

```
TOOLS = [{"type": "function", "function": {
    "name": "graph_deps",
    "description": "На какие положения транзитивно
        опирается документ (граф знаний, зависит_от+).",
    "parameters": {"type": "object",
        "properties": {"doc_id": {"type": "string",
                       "description": "например R65"}},
        "required": ["doc_id"]}}}, ...]

IMPL = {"search_words": search_words,
        "graph_deps": graph_deps,
        "check_rules": check_rules,
        "lean_check_claims": lean_check_claims}
```

Поле	Назначение
`name`	имя в вызове. Совпадает с ключом в `IMPL`
`description`	по нему модель решает, когда вызывать
`parameters`	схема аргументов
`required`	без чего вызов невозможен

`TOOLS` читает модель. `IMPL` читает программа. Имена должны совпадать.

Главное

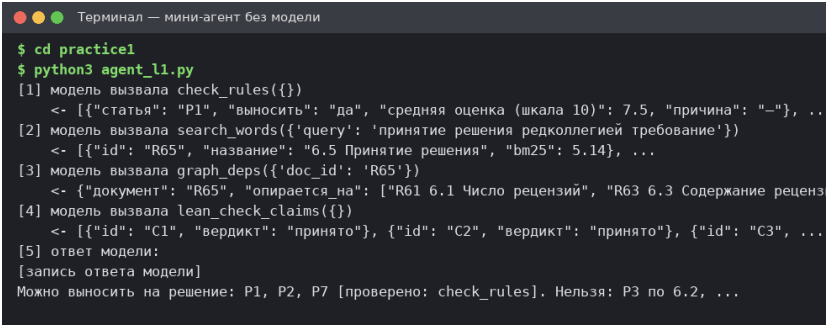
От качества описания зависит поведение агента. Пример значения в описании аргумента уменьшает число ошибочных вызовов.

Инструкция: каждая фраза против одной ошибки

Фраза инструкции	Зачем она
Ты ассистент редактора сборника.	роль и предмет разговора
Отвечай только по результатам инструментов.	против ответа по памяти модели: выдуманная оценка, вердикт «все восемь»
Каждую часть ответа помечай: [проверено: инструмент] или [найдено: инструмент].	ответ прослеживается: у каждого утверждения виден источник
Документы и пункты, которых нет в результатах инструментов, не называй.	против выдуманного стандарта
Если что-то не удалось подтвердить, пиши [не подтверждено].	честный выход вместо правдоподобной догадки
Идентификатор требования 6.5 в корпусе: R65.	сведение, которого модель не знает сама

- Проверка каждой строки: если её убрать, ошибётся ли агент? Если нет, строка лишняя.
- То же правило стоит в начале `AGENTS.md` вашего репозитория: инструкция агенту-помощнику устроена так же.

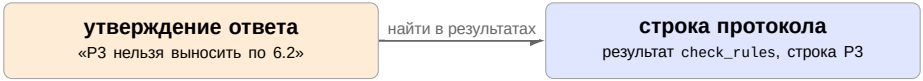
Запуск без модели



- Четыре результата со стрелкой настоящие. Порядок вызовов и текст ответа взяты из `PLAN` и `RECORD`.
- Так проверяются инструменты и цикл до подключения модели.

схема окна по документации, не снимок экрана

Как проверять ответ агента



- Каждое утверждение ответа ищется в результатах инструментов.
- Пометки в квадратных скобках подсказывают, где искать. Сами они проверкой не являются: их ставит модель.
- Утверждение, которого нет в результатах, порождено моделью и не проверено.
- С живой моделью порядок вызовов и слова будут другими. Способ проверки тот же.

Главное

Агент сам ничего не проверяет. Он выбирает инструмент и собирает ответ. Проверяют инструменты.

Как построить своего агента: семь шагов



- Шаги 1--5 выполняются без модели и занимают один вечер. Шаг 5 даёт работающую программу, по которой видно, хватает ли инструментов.
- Шаг 6 меняет одну строку: `llm = openai_llm`. Код инструментов и цикла не трогается.
- Файлы `agent_core.py` и `search_bm25.py` копируются в `src/` репозитория как есть.

Шаги 1--2: область и инструменты

```
corpus = pd.read_csv("kg/corpus.csv", dtype=str).fillna("")
rows = corpus.set_index("id")
bm25 = BM25((corpus["название"] + ". "
             + corpus["формулировка"]).tolist())

g = Graph()   # рёбра из столбца зависит_от, как в лекции 1

def search_words(query, k=5): ...
def graph_deps(doc_id): ...
def get_statement(doc_id):
    """Запись корпуса с источником."""
    r = rows.loc[doc_id]
    return {"id": doc_id, "тип": r["тип"],
            "название": r["название"],
            "формулировка": r["формулировка"],
            "зависит_от": [d for d in
                r["зависит_от"].split(";") if d],
            "источник": r["источник"]}
```

- Корпус есть `kg/corpus.csv` из задания 1. Два инструмента, поиск и граф, работают с любым корпусом этого формата.
- Третий инструмент `get_statement` возвращает запись с источником: правило «один вопрос, один источник» из `AGENTS.md`.
- Инструменты задачи редактора на свой корпус не переносятся. Их место займут проверки вашей области. Образец: `starter/src/agent_my.py`.

Шаги 3--5: описания, инструкция, заглушка

```
SYSTEM = ("Ты ассистент исследователя по корпусу "
  "утверждений <области>. Отвечай только по результатам "
  "инструментов. У каждого утверждения в ответе указывай "
  "id из корпуса. Помечай части ответа: [найдено: ...], "
  "[источник: ...], [не подтверждено].")

QUESTION = "<вопрос типа «зависимость» из README.md>"

PLAN = [("search_words", {"query": "..."}),
        ("get_statement", {"doc_id": "A14"}),
        ("graph_deps", {"doc_id": "A14"})]
RECORD = "[запись ответа модели]\n..."

llm = openai_llm if os.environ.get("LLM_API_KEY") \
      else scripted_llm(PLAN, RECORD)
run_agent(QUESTION, llm, SYSTEM, TOOLS, IMPL)
```

- В инструкции название области и сведения, которых модель не знает: обозначения, вид идентификаторов.
- Вопрос берётся из пяти вопросов исследователя: у него есть ожидаемый ответ, и ответ агента можно проверить.
- `RECORD` составляется по результатам инструментов, запущенных вручную. Так видно, хватает ли инструментов для ответа. Если нет, это повод добавить инструмент.

Добавить инструмент: три шага

1. **Функция.** Понятные имена аргументов, результат из простых значений, словарь с ключом «ошибка» при неверном аргументе.
2. **Описание в `TOOLS`.** Имя, назначение одной фразой, схема аргументов с примером значения.
3. **Запись в `IMPL`.** Имя из описания связывается с функцией.

Перед подключением функция вызывается вручную. На результат смотрят глазами модели: понятно ли, что это и откуда.

Какие инструменты появятся дальше

Лекция	Инструмент
2	`infer`, `check_consistency`: машина вывода
3	`validate_data`, `ready_articles`: проверка по формам, выводимый класс
4	`sparql_query`: запрос к графу
5--6	поиск по словам и по смыслу над своим корпусом
9--11	память агента и интерфейс MCP

Главное

Каждая лекция курса добавляет агенту инструмент. Цикл и модель остаются прежними.

Типичные ошибки при сборке агента

Ошибка	Что происходит и как исправить
имя в `TOOLS` и ключ в `IMPL` различаются	модель вызывает инструмент, программа возвращает `KeyError`. Сверить имена
инструмент возвращает объект, который не превращается в JSON	`json.dumps` падает. Возвращать строки, числа, списки, словари
результат на тысячи знаков	вытесняет остальное из окна модели. Обрезать, возвращать только нужное
описание инструмента из одного слова	модель не понимает, когда его вызывать. Назначение одной фразой и пример аргумента
инструкция без запрета называть лишнее	модель дополняет ответ из памяти. Фраза «не называй то, чего нет в результатах»
ключ в коде	ключ утекает в репозиторий. Только переменные окружения
инструмент меняет файлы	агент может испортить данные. Учебные инструменты только читают

Мини-агент в материалах курса

Где	Что
`13_РАЗБОР_агента_Л1_Л2.md`, `.docx`	пособие: пять файлов с разбором по строкам, протокол сообщений, восемь опытов, вопросы с ответами
`lecture2/agent/`	те же пять файлов, `Modelfile`, `run_local.sh` для прогона через Ollama
`practice1/`	агенты лекций 1 и 2, `ask.py` для вопросов из терминала, `llm_local.py`
`practice1/starter/src/agent_my.py`	образец агента над своим корпусом
`practice1/practice_1_editor.ipynb`	агент в блокноте с сохранённым протоколом
`lecture3/demo_lecture3.ipynb`, шаг 10	агент с проверкой данных по формам

Главное

Начинать с `python3 agent_l1.py` без модели. Затем `ask.py` с Ollama. Затем `agent_my.py` на своём корпусе.

Что сделать после занятия

Ollama

1. Установить, проверить `--version`.
2. `ollama pull qwen2.5:3b`.
3. `ollama create qwen-agent -f Modelfile`.
4. `python3 ask.py --status`.

Protégé

1. Скачать 5.6, запустить.
2. Открыть `review.owl`, пройти вкладки.
3. Запустить HermiT, найти жёлтые факты.
4. Внести ошибку, прочитать объяснение.

Мини-агент

1. `python3 agent_l1.py` без модели.
2. Прочитать `agent_core.py`: 50 строк.
3. Запустить `ask.py` с Ollama.
4. Собрать `agent_my.py` на своём корпусе.

Главное

Все три инструмента нужны для заданий курса. Без Ollama задания выполняются на записях, без Protégé онтология проверяется скриптом, без своего агента задание 1 не завершено.

Вопросы

oataeva@frccsc.ru khalov.a@phystech.edu